Modbus Server - 16377

Accessing robot data through the MODBUS server.

Examples are valid for:
CB2 Software version: 1.6 and forward
CB3 Software version: All versions
Note that older or newer software versions may behave differently.

Purpose:

Give read and write access to data in the robot controller for other devices

How it works:

The robot controller acts as a Modbus TCP server (port 502), clients can establish connections to it and send standard MODBUS requests to it. The server is available at the IP address of the controller which can be found and modified in PolyScope (SETUP Robot → Setup NETWORK) For more information about MODBUS, see www.modbus.org (http://www.modbus.org/) specifications of MODBUS messages can be found at http://www.modbus.org/docs/Modbus_Application_Protocol_V1_1b.pdf (http://www.modbus.org/docs/Modbus_Application_Protocol_V1_1b.pdf)

Note:
Some Modbus device manufacturers use the terms Master (client) and Slave (server). Typically the external IO device is going to be a server and the robot is going to behave as the client (requesting and consuming messages from the server). Master=client and Slave=server. However, note that the UR controller can be both a server and a client

Functionality:

- Several clients can connect to the server at the same time
- The server can respond to the following function codes:

  - 0x01: READ_COILS (read output bits)
  - 0x02: READ_DISCRETE_INPUTS (read input bits)
  - 0x03: READ_HOLDING_REGISTERS (read output registers)
  - 0x04: READ_INPUT_REGISTERS (read input registers)
  - 0x05: WRITE_SINGLE_COIL (write output bit)
  - 0x06: WRITE_SINGLE_REGISTER(write output register)
  - 0x0F: WRITE_MULTIPLE_COILS (write multiple output bits)
  - 0x10: WRITE_MULTIPLE_REGISTERS (write multiple output registers)

- The server will send a response to all requests
- The server responds error messages with exception codes:

- 0x01: ILLEGAL_FUNCTION_CODE
- 0x02: ILLEGAL_DATA_ACCESS (if the request address is illegal)
- 0x03: ILLEGAL_DATA_VALUE (if the request data is invalid)

- The 16 bit ports can be read using the script function read_port_register(<address>), many of the ports, including 128 general purpose registers (address 128-256) (see the port map) can also be written to using write_port_register(<address>,<value>). You can use the script function integer_to_binary_list(<value>) to get the register value as a list of booleans (e.g.[True,False,.....]), or binary_list_to_integer(<list>) to go back, please refer to the URScript manual for details.

- Coils (digital ports) can be accessed using read_port_bit(<address>) and write_port_bit(<address>,<value>).

- Note that all values are unsigned, if you want to convert to signed integers, program "if (val > 32768): val = val - 65535".

- The MODBUS Server has 0-based addressing. Be aware that some other devices are 1-based (e.g. Anybus X-gateways), then just add one to the address on that device. (e.g. address 3 on the robot will be address 4 on the Anybus X-gateway)

Setup:

Setup the controller IP and your MODBUS client with static IP addresses in the same subnet, see Set up Modbus communication - 16357 (/how-tos-and-faqs/how-to/ur-how-tos/modbus-communication-16357/)

Port map 16bit register addresses:

- Use function codes 0x03: READ_HOLDING_REGISTERS, 0x04: READ_INPUT_REGISTERS, 0x06: WRITE_SINGLE_REGISTER, and 0x10: WRITE_MULTIPLE_REGISTERS to access the following addresses.
- Use the script functions read_port_register(<address>) and write_port_register(<address>, <value>) to read and write to those addresses on your robot from the running program.

Please note that some registers has been added and the list below has been marked what registers is avalable from what version.

R=Read, W=Write

Address	Pre 3.0	3.0	3.1	R	W	
0	x	x	x	*		Inputs, bits 0-15 [BBBBBBBBTTxxxxxx] x=undef, T=tool, B=box
1	x	x	x	*	*	Outputs, bits 0-15 [BBBBBBBBTTxxxxxx] x=undef, T=tool, B=box
2	x	x	x		*	SetOutputsBitsMask 0-15 [BBBBBBBBTTxxxxxx] x=undef, T=tool, B=box
3	x	x	x		*	ClearOutputsBitsMask 0-15 [BBBBBBBBTTxxxxxx] x=undef, T=tool, B=box
4	x	x	x	*		Analog input 0 (0-65535)
						Analog input 0 domain e {0=current[mA], 1=voltage[mV]}

5	x	x	x	*	*	
6	x	x	x	*		Analog input 1 (0-65535)
7	x	x	x	*	*	Analog input 1 domain e {0=current[mA], 1=voltage[mV]}
8	x	x	x	*		Analog input 2 (tool) (0-65535)
9	x	x	x	*	*	Analog input 2 (tool) domain e {0=current[mA], 1=voltage[mV]}
10	x	x	x	*		Analog input 3 (tool) (0-65535)
11	x	x	x	*	*	Analog input 3 (tool) range e {0=current[mA], 1=voltage[mV]}
16	x	x	x	*	*	Analog output 0 output (0-65535)
17	x	x	x	*	*	Analog output 0 output domain e {0=current[mA], 1=voltage[mV]}
18	x	x	x	*	*	Analog output 1 output (0-65535)
19	x	x	x	*	*	Analog output 1 output domain {0=current[mA], 1=voltage[mV]}
20	x	x	x	*	*	Tool output voltage (V) e {0V, 12V, 24V}
21	x	x	x	*		Tool digital input bits
22	x	x	x	*	*	Tool digital output bits
24	x	x	x	*		Euromap67 input bits (0-15)
25	x	x	x	*		Euromap67 input bits (16-32)
26	x	x	x	*		Euromap67 output bits (0-15) (read only!)
27	x	x	x	*		Euromap67 output bits (16-32) (read only!)
28	x	x	x	*		Euromap 24V voltage
29	x	x	x	*		Euromap 24V current
30	-	x	x	*		Configurable inputs, bits [BBBBBBBBxxxxxxxx] x=undef, T=tool, B=box
31	-	x	x	*	*	Configurable outputs, bits [BBBBBBBBxxxxxxxx] x=undef, T=tool, B=box
32	-	x	x		*	Bit mask configurable outputs, bits [BBBBBBBBxxxxxxxx] x=undef, T=tool, B=box
33	-	x	x		*	Clear configurable outputs, bits [BBBBBBBBxxxxxxxx] x=undef, T=tool, B=box
34-127	x	x	x			Reserved for future system variables
128-255	x	x	x	*	*	General purpose 16 bit registers
256-	x	x	x	*		Robot state
512-	x	x	x	*		Program state (not implemented)
768-	x	x	x	*		Tool states
1024-	x	x	x	*		GUI state (not implemented)
2048-	x	x	-	*		RT Machine control

Robot state

256	x	x	x	*	Controller version high nuber		
257	x	x	x	*	Controller version low number		
258	x	x	x	*	Robot mode		
						CB3 and 3.1	Disconnected=0, Confirm_safety=1, Booting=2, Power_off=3, Power_on=4, Idle=5, Backdrive=6, Running=7
						CB2	No_controller=-1, Running=0, Freedrive=1,Ready=2, Initializing=3, Security_stoppde=4, Emergency_stopped=5, Fault=6, Not_connected=8, Shoutdown=9
260	x	x	x	*	isPowerOnRobot		
261	x	x	x	*	isSecurityStopped		
262	x	x	x	*	isEmergencyStopped		
263	x	x	x	*	isTeachButtonPressed		
264	x	x	x	*	isPowerPuttonPressed		
265	x	x	x	*	isSafetySignalSuchThatWeShouldStop		
270	x	x	x	*	Base joint angle (in mrad)		
271	x	x	x	*	Shoulder joint angle (in mrad)		
272	x	x	x	*	Elbow joint angle (in mrad)		
273	x	x	x	*	Wrist1 joint angle (in mrad)		
274	x	x	x	*	Wrist2 joint angle (in mrad)		
275	x	x	x	*	Wrist3 joint angle (in mrad)		
280	x	x	x	*	Base joint angle velocity (in mrad/s)		
281	x	x	x	*	Shoulder joint angle velocity (in mrad/s)		
282	x	x	x	*	Elbow joint angle velocity (in mrad/s)		
283	x	x	x	*	Wrist1 joint angle velocity (in mrad/s)		
284	x	x	x	*	Wrist2 joint angle velocity (in mrad/s)		
285	x	x		*	Wrist3 joint angle velocity (in mrad/s)		
290	x	x	x	*	Base joint current (in mA)		
291	x	x	x	*	Shoulder joint current (in mA)		
292	x	x	x	*	Elbow joint current (in mA)		

293	x	x	x	*	Wrist1 joint current (in mA)
294	x	x	x	*	Wrist2 joint current (in mA)
295	x	x	x	*	Wrist3 joint current (in mA)
300	x	x	x	*	Base joint temperature (in C)
301	x	x	x	*	Shoulder joint temperature (in C)
302	x	x	x	*	Elbow joint temperature (in C)
303	x	x	x	*	Wrist1 joint temperature (in C)
304	x	x	x	*	Wrist2 joint temperature (in C)
305	x	x	x	*	Wrist3 joint temperature (in C)

Joint modes From version 1.7

310	x	x	x	*	Base joint mode
311	x	x	x	*	Shoulder joint mode
312	x	x	x	*	Elbow joint mode
313	x	x	x	*	Wrist1 joint mode
314	x	x	x	*	Wrist2 joint mode
315	x	x	x	*	Wrist3 joint mode

List of Joint Modes

JOINT_SHUTTING_DOWN_MODE = 236;

JOINT_PART_D_CALIBRATION_MODE = 237;

JOINT_BACKDRIVE_MODE = 238;

JOINT_POWER_OFF_MODE = 239;

JOINT_NOT_RESPONDING_MODE = 245;

JOINT_MOTOR_INITIALISATION_MODE = 246;

JOINT_BOOTING_MODE = 247;

JOINT_PART_D_CALIBRATION_ERROR_MODE = 248;

JOINT_BOOTLOADER_MODE = 249;

JOINT_CALIBRATION_MODE = 250;

JOINT_FAULT_MODE = 252;

JOINT_RUNNING_MODE = 253;

JOINT_IDLE_MODE = 255;

320	-	-	x	*	Base joint revolution count (number of full turns, typically 0 or 1)
321	-	-	x	*	Shoulder joint revolution count
322	-	-	x	*	Elbow joint revolution count
323	-	-	x	*	Wrist1 joint revolution count
324	-	-	x	*	Wrist2 joint revolution count

325	-	-	x	*	Wrist3 joint revolution count
TCP					
400	x	x	x	*	TCP-x in tenth of mm (in base frame)
401	x	x	x	*	TCP-y in tenth of mm (in base frame)
402	x	x	x	*	TCP-z in tenth of mm (in base frame)
403	x	x	x	*	TCP-rx in mrad (in base frame)
404	x	x	x	*	TCP-ry in mrad (in base frame)
405	x	x	x	*	TCP-rz in mrad (in base frame)
410	x	x	x	*	TCP-x speed in mm/s (in base frame)
411	x	x	x	*	TCP-y speed in mm/s (in base frame)
412	x	x	x	*	TCP-z speed in mm/s (in base frame)
413	x	x	x	*	TCP-rx speed in mrad/s (in base frame)
414	x	x	x	*	TCP-ry speed in mrad/s (in base frame)
415	x	x	x	*	TCP-rz speed in mrad/s (in base frame)
420	x	x	x	*	TCP-x offset in mm (in tool frame)
421	x	x	x	*	TCP-y offset in mm (in tool frame)
422	x	x	x	*	TCP-z offset in mm (in tool frame)
423	x	x	x	*	TCP-rx offset in mrad (in tool frame)
424	x	x	x	*	TCP-ry offset in mrad (in tool frame)
425	x	x	x	*	TCP-rz offset in mrad (in tool frame)
450	x	x	x	*	Robot current (in mA)
451	x	x	x	*	I/O current (in mA)
768	x	x	x	*	Tool state
769	x	x	x	*	Tool temperature (in C)
770	x	x	x	*	Tool current (in mA)
GUI state					
1024	x	x	x	*	GUI state not implemented

Port map bit (Digital) addresses

- Use function codes 0x01: READ_COILS, 0x02: READ_DISCRETE_INPUTS, 0x05: WRITE_SINGLE_COIL, 0x0F: WRITE_MULTIPLE_COILS to access the following addresses.
- Use the script functions read_port_bit(<address>) and write_port_bit(<address>, <value>) to read and write to those addresses on your robot from the running program.

Address	Pre 3.0	3.0	3.1	R	W	
0-15	x	x	x	*		Inputs, bits 0-15 [BBBBBBBBTTxxxxxx] x=undef, T=tool, B=box
16-31	x	x	x	*	*	Outputs, bits 0-15 [BBBBBBBBTTxxxxxx] x=undef, T=tool, B=box
32-47	x	x	x		*	SetOutputsBitsMask 0-15 [BBBBBBBBTTxxxxxx] x=undef, T=tool, B=box
48-63	x	x	x		*	ClearOutputsBitsMask 0-15 [BBBBBBBBTTxxxxxx] x=undef, T=tool, B=box
64-79	x	x	x	*		Euromap67 input bits (0-15)
80-95	x	x	x	*		Euromap67 input bits (16-32)
96-111	x	x	x	*		Euromap67 output bits (0-15) (read only!)
112-127	x	x	x	*		Euromap67 output bits (16-32) (read only!)
128-135	x	x	x	*		Configurable inputs, bits [BBBBBBBBxxxxxxxx] x=undef, T=tool, B=box
136-143	x	x	x	*	*	Configurable outputs, bits [BBBBBBBBxxxxxxxx] x=undef, T=tool, B=box
144-151	x	x	x		*	Bit mask configurable outputs, bits [BBBBBBBBxxxxxxxx] x=undef, T=tool, B=box
152-159	x	x	x		*	Clear configurable outputs, bits [BBBBBBBBxxxxxxxx] x=undef, T=tool, B=box
260	x	x	x	*		isPowerOnRobot
261	x	x	x	*		isProtectiveStopped
262	x	x	x	*		isEmergencyStopped
263	x	x	x	*		isTeachButtonPressed
264	x	x	x	*		isPowerPuttonPressed
265	x	x	x	*		isSafetySignalSuchThatWeShouldStop

Important info for Server applications with Siemens clients(masters)

From UR Software version 1.7 and on:

Some MODBUS units use designated addresses for each function code, i.e. Siemens use: coils (generally addressed as 0xxxx), contacts (1xxxx), input registers (3xxxx), holding registers (4xxxx). To solve this issue, address "x" is reflected at 10000+"x", 20000+"x", 30000+"x" and 40000+"x" Address 0 is also reflected at address 9999, as some units have an addresses starting at 1

The value "Unit Identifier"/"Slave ID" is ignored in the MODBUS Server